Dijkstras Algorithm

AlgoExpert · Hard · Famous Algorithms

The problem

- Edges[v] lists [destination, weight] (non-negative).
- Shortest distance from start to every vertex, -1 if unreachable.

Intuition

- Always expand the closest unvisited vertex; with non-negative weights its distance is final.

Approach

- 1. Distances start at infinity
- 2. Use a min-heap of distance and vertex
- 3. Relax outgoing edges
- 4. Unreached vertices are minus one

Java solution

```java
public class DijkstrasAlgorithm {
    public static int[] dijkstrasAlgorithm(int start, int[][][] edges) {
        int n = edges.length;
        int[] dist = new int[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[start] = 0;
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Integer.compare(a[1], b[1]));
        pq.add(new int[]{start, 0});
        while (!pq.isEmpty()) {
            int[] c = pq.poll();
            if (c[1] > dist[c[0]]) continue;
            for (int[] e : edges[c[0]]) {
                int nd = c[1] + e[1];
                if (nd < dist[e[0]]) { dist[e[0]] = nd; pq.add(new int[]{e[0], nd}); }
            }
        }
        for (int i = 0; i < n; i++) if (dist[i] == Integer.MAX_VALUE) dist[i] = -1;
        return dist;
    }
}
```

Complexity

- Time: O((v + e) log v)
- Space: O(v)

Recap

- Always expand the closest unvisited vertex; with non-negative weights its distance is final.
- Complexity: O((v + e) log v)